

Research Report on Aero-engine Predictive Maintenance Based on Physical Intuition and Neural Learning

NUOHU LI QIUYU WANG ZIRUI ZHANG

April 29, 2026

Abstract. This project studies predictive maintenance for turbofan engines using the NASA C-MAPSS FD001 degradation dataset. The central goal is not only to classify whether an engine is entering a high-risk failure window, but also to explain why the model reaches that warning decision. The workflow combines statistical learning ideas, including variance diagnostics, correlation analysis, PCA, sliding-window feature engineering, and a Random Forest baseline, with a neural-learning design based on LSTM-style temporal modeling and risk-weighted classification. The final interpretation emphasizes missed-alarm risk, physical sensor degradation patterns, and the limits of judging industrial models only by aggregate predictive accuracy.

1 Introduction and Research Motivation

Predictive maintenance is a central application of machine learning in industrial systems because the cost of a wrong decision is asymmetric. In an aero-engine setting, a false alarm may lead to an unnecessary inspection, but a missed alarm can lead to catastrophic equipment damage, unplanned downtime, and safety risk. Therefore, a useful predictive maintenance model should be evaluated not only by overall accuracy, but also by whether it reduces false negatives and whether its decisions are consistent with the physical process of degradation.

This report focuses on the NASA C-MAPSS turbofan engine degradation simulation data, specifically the FD001 subset. The dataset contains simulated engine life-cycle records, operating settings, and sensor measurements. NASA describes C-MAPSS data as high-fidelity simulated data designed to represent nominal and fault engine degradation over a series of flights, and the FD001 subset is commonly used as a benchmark for Remaining Useful Life (RUL) and early-warning modeling [1, 2]. In this project, the original RUL prediction task is converted into a binary early-warning task: an engine is labeled as high-risk if it is within 30 cycles of failure, and healthy otherwise.

The project intentionally integrates multiple learning paradigms. From the statistical-learning side, it uses exploratory diagnostics, feature filtering, correlation analysis, PCA, and a Random Forest baseline with slope-based high-order features. From the deep-learning side, the proposed architecture is an Attention-LSTM model with an asymmetric focal loss designed to penalize missed faults more heavily than false alarms. In the executed notebook environment, TensorFlow was unavailable, so the executable neural comparison used a compact scikit-learn MLP fallback. The Attention-LSTM design is therefore treated as the proposed architecture, while the reported executable ROC comparison is interpreted as a validation of the broader statistical-plus-neural workflow.

2 Data Source and Problem Definition

2.1 Dataset

The data source is the NASA C-MAPSS Turbofan Engine Degradation Simulation Dataset, with this report using FD001. The raw files used in the analysis are `train_FD001.txt`, `test_FD001.txt`, and `RUL_FD001.txt`. The training file contains run-to-failure trajectories: each engine is observed over time until the end of its recorded operating life. The test file contains truncated trajectories, and the RUL file provides the true remaining life values needed to align the test labels.

Each observation contains an engine unit identifier, a cycle number, three operating settings, and 21 sensor readings. The raw training data used in the original report contains 20,631 observations and 26 columns: two index/time variables, three operational settings, and 21 sensor channels. The prediction target is not directly available as a single column in the raw training file, so it must be constructed from the life-cycle structure of each engine.

2.2 Response Variable and Explanatory Variables

For a training observation at cycle t in an engine whose final observed cycle is $T_{\max}$, Remaining Useful Life is defined as

$$RUL_t = T_{\max} - t. \quad (1)$$

The binary response variable is then

$$y_t = \begin{cases} 1, & RUL_t \leq 30, \\ 0, & RUL_t > 30. \end{cases} \quad (2)$$

Here, $y_t = 1$ means the engine is in the danger zone and should trigger an early-warning decision. The 30-cycle threshold is a practical modeling choice: it turns a continuous RUL problem into an industrially interpretable alarm/no-alarm classification problem.

Table 1: Main modeling variables and their roles.

Group	Variables / Construction	Role in the analysis
Response	Binary label from RUL: 1 if $RUL \leq 30$, otherwise 0	Final classification target for early-warning maintenance.
Intermediate target	RUL from life-cycle endpoint and current cycle	Used to build the binary response, not the final model output.
Index / time	<code>unit_number</code> , <code>time_cycles</code>	Preserve engine identity and temporal ordering for window construction.
Operating settings	<code>setting_1</code> , <code>setting_2</code> ; <code>setting_3</code> removed after variance diagnostics	Contextual explanatory variables after filtering redundant signals.
Sensors	Active sensor readings after removing near-constant channels	Main physical inputs representing engine degradation.
Window features	30-cycle sequences and local slopes	Capture recent temporal history and the rate of physical change.

3 Exploratory Analysis and Physical Feature Filtering

The first modeling step is to remove information that is unlikely to help classify degradation. Variance diagnostics identified several nearly constant channels: `sensor_1`, `sensor_5`, `sensor_10`, `sensor_16`, `sensor_18`, `sensor_19`, and `setting_3`. These variables remain close to flat over the engine life cycle, so they add computational burden without contributing meaningful degradation information. Removing them reduces noise and helps prevent the model from learning artificial patterns from constant or near-constant features.

The second step is to check how active sensors move with operating time. Pearson correlation coefficients showed a clear split between sensors whose values increase with wear and sensors whose values decrease with wear. Strong positive correlations appear in sensors such as `sensor_11`, `sensor_4`, `sensor_15`, and `sensor_17`. Strong negative correlations appear in sensors such as `sensor_12`, `sensor_7`, `sensor_21`, and `sensor_20`. This pattern matters because it supports a physically grounded interpretation: failure risk is not only a statistical label, but a signal emerging from directional changes in sensor behavior.

The visualization of Engine Unit 1 further supports this physical intuition. A positively correlated sensor rises over time, while a negatively correlated sensor declines. The late-life period shows more severe fluctuation and sharper trend changes, which is consistent with the idea that engines become less stable near failure. This motivates a temporal model: a single observation may not be enough, because the recent pattern of sensor movement is more informative than the current value alone.

4 Feature Engineering

4.1 RUL Transformation and Early-warning Labeling

The transformation from RUL to a binary label is a deliberate modeling decision. Continuous RUL prediction is more detailed, but it can be difficult to use directly in maintenance planning because small numerical errors may not correspond to different operational choices. A binary warning task is simpler: the model asks whether the equipment is already close enough to failure to justify intervention. This choice also matches the asymmetric-risk logic of the project, where missing a true danger event is more costly than issuing an early inspection.

4.2 Physical Data Augmentation

Industrial sensors often face short-term disturbances from vibration, electromagnetic noise, or measurement instability. To reflect this, the project introduces a micro-fault injection mechanism during training. Random, short-duration noise is inserted into selected sensor channels for a small number of time steps. The purpose is not to create artificial labels, but to make the model less fragile when it encounters noisy sensor behavior. Importantly, normalization is fitted before the noise-injection transformation contaminates the scale parameters. This helps avoid letting artificial outliers determine the normalization basis.

4.3 Sliding-window Reconstruction

Time-series modeling requires recent history. The report uses 30-cycle sliding windows. For the Random Forest baseline, each window is flattened and then supplemented with local slope features. The slope features act as an explicit estimate of the rate of degradation. This is important because a Random Forest cannot naturally process three-dimensional temporal sequences; slope engineering gives the static model access to temporal dynamics.

For the neural model design, each window is represented as a three-dimensional tensor with shape $\text{samples} \times \text{time steps} \times \text{features}$. This format is suitable for LSTM-style temporal modeling, because the model can process ordered sensor sequences rather than treating all features as independent static columns.

4.4 PCA as a Diagnostic, Not the Final Model

PCA is used as a diagnostic visualization rather than as the final predictive solution. When healthy and high-risk samples are projected into two principal components, the two groups overlap substantially. This does not prove that no model can separate them; instead, it suggests that a

simple linear boundary in low-dimensional space is insufficient. The overlap justifies using nonlinear models and temporal information rather than relying only on static linear separation.

5 Modeling Strategy

5.1 High-order Random Forest Baseline

The Random Forest baseline combines active sensor values, flattened sliding-window features, and local degradation slopes. It is a strong baseline because tree-based methods can capture nonlinear interactions and do not require strict distributional assumptions. In this project, the Random Forest is not treated as a weak comparison. In fact, because the slope features encode important physical information, the Random Forest achieves very strong ROC performance.

However, the baseline remains limited because it receives engineered summaries rather than learning the full temporal dependency structure directly. It can use local slopes, but it does not truly model sequential order in the same way as an LSTM. This is the main reason for proposing a neural temporal architecture.

5.2 Proposed Attention-LSTM Architecture

The proposed deep-learning design uses a two-layer LSTM followed by a temporal attention layer. LSTM layers are appropriate for sequence data because they can learn dependencies across time steps. The attention layer is designed to assign different weights to different moments within the 30-cycle window. In a physical maintenance problem, this is especially useful because not every part of the window is equally informative. If the engine is near failure, the most recent abnormal sensor behavior may deserve more weight than older stable readings.

Mathematically, the attention mechanism first produces a score for each hidden state h_t :

$$e_t = \tanh(W_a h_t + b_a). \quad (3)$$

The scores are normalized by a softmax function:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^T \exp(e_k)}. \quad (4)$$

The final context vector is then a weighted sum:

$$V = \sum_{t=1}^T \alpha_t h_t. \quad (5)$$

This design supports interpretability because the attention weights can be inspected to see which time steps influenced the warning decision.

5.3 Asymmetric Focal Loss

The proposed model uses an asymmetric focal loss to reflect the cost structure of predictive maintenance. Standard accuracy treats false positives and false negatives too symmetrically. In aero-

engine maintenance, a missed fault is much worse than an unnecessary warning. The project therefore assigns a larger penalty to false negatives than false positives:

$$L = -\frac{1}{N} \sum_{i=1}^N [\gamma_{fn}(1 - \hat{y}_i)^\alpha y_i \log(\hat{y}_i) + \gamma_{fp}\hat{y}_i^\alpha (1 - y_i) \log(1 - \hat{y}_i)]. \quad (6)$$

The intended configuration uses $\alpha = 2$, $\gamma_{fn} = 10$, and $\gamma_{fp} = 1$. This means the training objective is deliberately conservative: the model is pushed to avoid missed alarms, even if that increases false alarms.

5.4 Executed Neural Fallback

A practical limitation occurred in the executed notebook: TensorFlow was unavailable in the kernel used for the final executable run. As a result, the Attention-LSTM implementation was defined as the intended architecture, but the plotted executable neural comparison used a compact MLP fallback from scikit-learn. This should be stated clearly because it affects interpretation. The neural result demonstrates that a neural-learning baseline can be integrated into the pipeline, but it should not be overclaimed as a fully executed Attention-LSTM result unless TensorFlow/Keras is installed and the LSTM cells are rerun.

6 Model Evaluation and Results

The evaluation uses precision, recall, F1-score, false negatives, and ROC-AUC. For this problem, recall for the high-risk class is the most safety-critical metric because it measures how many true danger cases are detected. Precision is still relevant, since false alarms waste maintenance resources, but in this context precision should not dominate recall.

Table 2: Executed model comparison from the notebook environment.

Model	Precision	Recall	F1	AUC	Interpretation
Random Forest with slope features	0.88–0.90	0.74	0.80–0.81	≈ 0.997	Strong overall discrimination, but lower high-risk recall.
Fallback MLP neural baseline	0.80	0.82	0.81	≈ 0.996	Slightly lower precision, but better detection of high-risk cases.

The Random Forest has excellent overall discrimination and high precision, which means that when it raises a warning, it is usually correct. However, its high-risk recall is lower than the neural fallback, meaning it misses more danger cases. The fallback MLP improves high-risk recall, which aligns better with the maintenance objective, though its precision is lower than the Random Forest. This is an important result: the best model in an industrial safety setting is not always the model with the highest precision or the cleanest aggregate accuracy. The model must be judged by the cost of its mistakes.

The ROC curves show that both models separate healthy and high-risk periods extremely well. However, ROC-AUC alone can hide the practical difference between a false alarm and a missed

fault. A model can have very high AUC while still operating at a threshold that misses too many true danger cases. For deployment, the final threshold should therefore be selected based on a cost function such as

$$Cost = C_{FP}N_{FP} + C_{FN}N_{FN}, \quad (7)$$

where C_{FN} should be much larger than C_{FP} in an aero-engine maintenance setting.

7 Interpretability and Critical Reasoning

Interpretability is central to this project because predictive maintenance models must be trusted by engineers. The exploratory analysis gives the first layer of interpretability: active sensors show consistent positive or negative movement with engine wear. The sliding-window design gives the second layer: the model is not predicting from a single noisy observation, but from recent degradation patterns. The proposed attention mechanism gives the third layer: if the Attention-LSTM is executed, attention weights can show whether the model concentrates on late-window signals that are physically closer to the current decision point.

The PCA figure also supports critical reasoning. Healthy and high-risk samples overlap in a two-dimensional PCA projection, which warns against overinterpreting simple visual separation. If a model performs well, it is likely because it uses nonlinear combinations, time-window structure, or high-order degradation slopes, not because the raw data are trivially separable. This is exactly why the project combines statistical diagnostics with machine-learning modeling.

At the same time, the project has limitations. First, FD001 represents a relatively controlled operating condition. Real aircraft engines may experience multiple operating regimes, environmental changes, and multiple fault modes. Second, micro-fault injection is a useful robustness idea, but it is still synthetic. Third, the final executed neural comparison used an MLP fallback because TensorFlow was unavailable. Therefore, the Attention-LSTM should be considered a proposed extension unless the deep-learning environment is installed and the model is rerun. Finally, the current models output point predictions; uncertainty quantification would make the warning system more useful for decision support.

8 Data and Software Sources

The dataset used in this project is the NASA C-MAPSS Turbofan Engine Degradation Simulation Dataset, FD001 subset. NASA’s public dataset documentation identifies the C-MAPSS simulator data as engine degradation simulation data and cites the PHM 2008 damage propagation modeling reference [1, 2]. The raw files used were `train_FD001.txt`, `test_FD001.txt`, and `RUL_FD001.txt`.

The analysis was conducted in Python. The main packages used were pandas for structured data handling [3]; NumPy for numerical array operations [4]; SciPy for slope/statistical computation [5]; scikit-learn for scaling, PCA, Random Forest, MLP fallback, and evaluation metrics [6]; Matplotlib and seaborn for visualization [7, 8]; and TensorFlow/Keras for the intended Attention-LSTM architecture when available [9, 10].

Table 3: Software packages cited for reproducibility.

Package	Use in this project
pandas	Loaded raw text files, assigned semantic column names, managed engine-level grouping, and organized derived labels.
NumPy	Supported numerical array operations, tensor reshaping, random perturbation, and vectorized computations.
SciPy	Supported local slope calculations used in high-order Random Forest feature engineering.
scikit-learn	Implemented scaling, PCA, Random Forest, MLP fallback, classification reports, ROC curves, and AUC metrics.
Matplotlib / seaborn	Generated the correlation, degradation trajectory, PCA, and ROC visualizations.
TensorFlow / Keras	Defined the intended Attention-LSTM, temporal attention layer, and asymmetric focal loss when the runtime environment supports it.

Listing the packages directly in the report is important because the submitted code is not the only source of reproducibility. A reader should be able to understand which tools produced the data loading, model fitting, metrics, and figures even without reading every notebook cell. This also addresses the project instruction to cite all data sources and software packages used.

9 Conclusion

This project shows that predictive maintenance should be framed as a risk-sensitive decision problem rather than a pure accuracy-maximization problem. The statistical-learning components identify informative sensors, remove redundant features, reveal nonlinear overlap, and create physically meaningful slope features. The neural-learning component provides a path toward direct temporal modeling and attention-based interpretability. Together, these methods support the assignment goal of integrating multiple learning paradigms while emphasizing interpretation and critical reasoning.

The most important practical conclusion is that false negatives matter more than false positives in aero-engine maintenance. A model that slightly sacrifices precision may still be preferable if it substantially improves high-risk recall. The final system should therefore be evaluated by maintenance cost, missed-fault reduction, and physical credibility, not only by AUC or overall accuracy. Future work should rerun the full Attention-LSTM in a TensorFlow-enabled environment, test cross-condition datasets such as FD002 or FD004, and add uncertainty quantification for safer maintenance decisions.

References

- [1] NASA. *C-MAPSS Jet Engine Simulated Data*. NASA Open Data Portal.
- [2] Saxena, A., Goebel, K., Simon, D., and Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. *Proceedings of the 1st International Conference on Prognostics and Health Management*.

- [3] McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*.
- [4] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- [5] Virtanen, P., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.
- [6] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [7] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- [8] Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.
- [9] Abadi, M., et al. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org.
- [10] Chollet, F., et al. *Keras: Deep Learning for Humans*. Keras project documentation.

A Appendix: Visual Evidence from the Executed Code

The appendix preserves the main visual evidence while keeping the main report within the requested page range. These figures support the modeling argument: sensors move with engine wear, individual engines show late-life degradation, healthy and high-risk samples overlap in PCA space, and both models show strong ROC behavior.

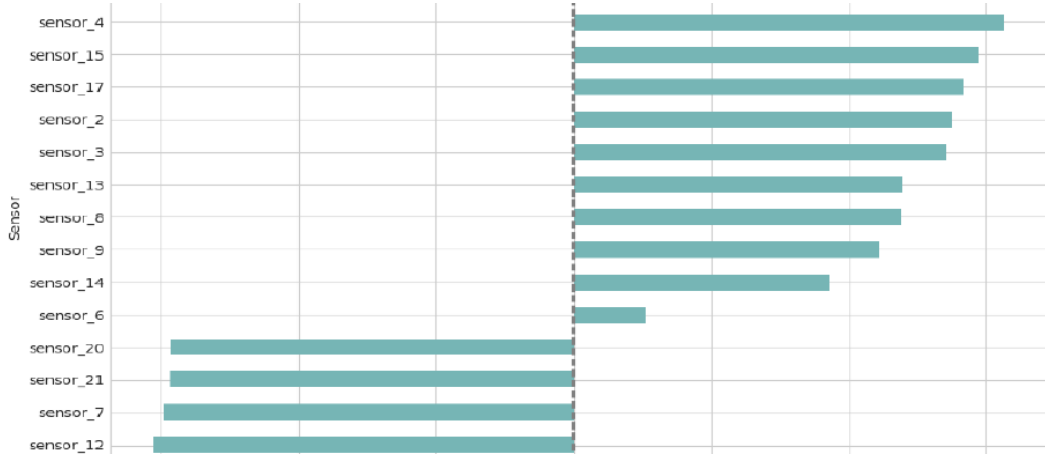


Figure 1: Correlation between each active sensor and operating time. Positive values indicate sensors that increase with wear; negative values indicate sensors that decline near failure.

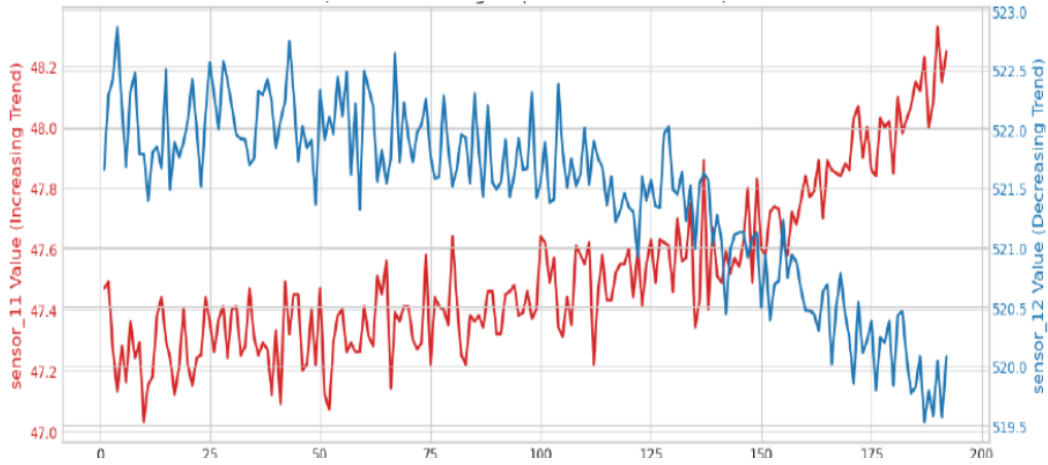


Figure 2: Full-life-cycle degradation trajectory for Engine Unit 1, comparing a positively correlated sensor and a negatively correlated sensor.

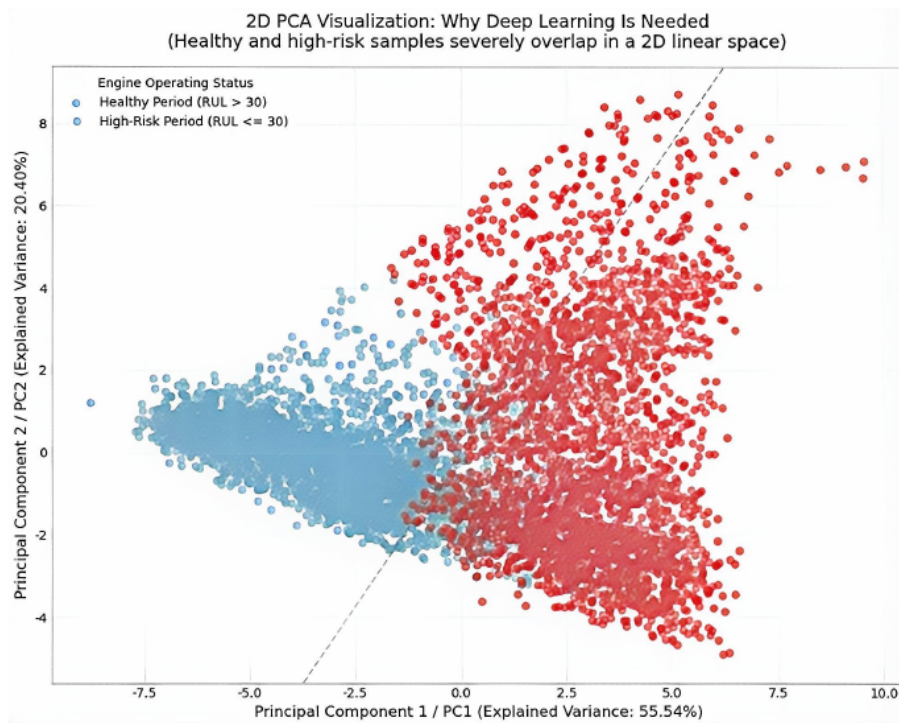


Figure 3: Two-dimensional PCA visualization. The overlap between healthy and high-risk samples supports the need for nonlinear and temporal modeling.

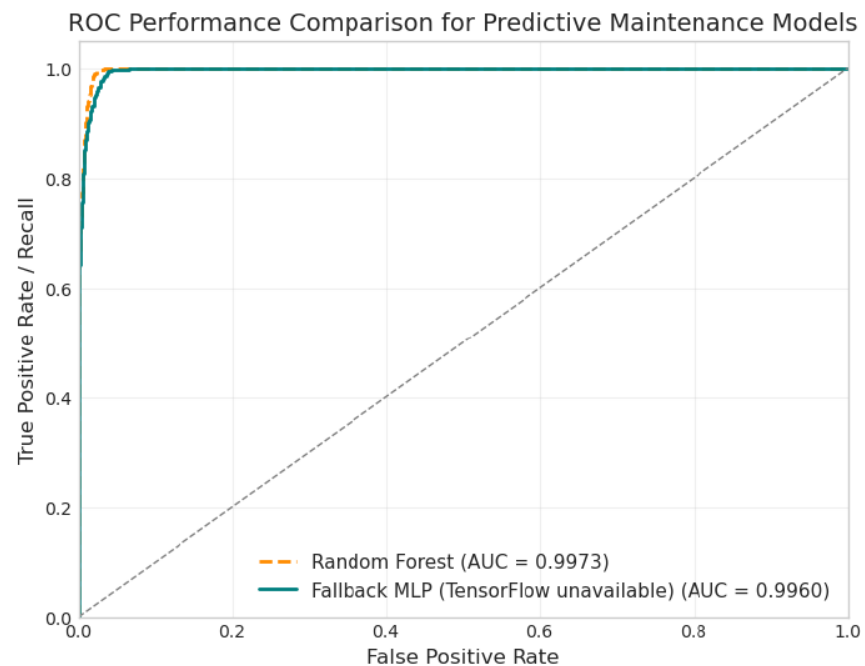


Figure 4: ROC comparison from the executed notebook output. Since TensorFlow was unavailable in that run, the neural curve corresponds to the fallback MLP rather than the full Attention-LSTM.